

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Proyecto Fase 3

Índice

1. Resumen	4
2. Objetivos	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	4
3. Enunciado	4
3.1 Alcance específico de la Fase 3	4
3.2 Panel administrativo: cupones y devoluciones	5
3.2.1 Reglas generales del panel (obligatorio)	5
3.2.2 Cupones (obligatorio)	5
3.2.3 Devoluciones (obligatorio)	6
3.3 Cron y notificaciones	7
3.3.1 Recordatorio de carrito (cron obligatorio)	7
3.3.2 Notificación de stock bajo a proveedor (obligatorio)	7
3.3.3 Notificación post-compra al cliente (obligatorio)	7
3.4 Observabilidad: métricas, logs, dashboards y alertas	8
3.4.1 ELK Stack (obligatorio)	8
3.4.2 Prometheus (obligatorio)	8
3.4.3 Grafana (obligatorio)	9
3.5 Infraestructura como código (Terraform + Ansible)	9
3.5.1 Regla principal (obligatoria)	9
3.5.2 Terraform: infraestructura obligatoria	9
3.5.3 Estructura mínima del repo Terraform (obligatorio)	10
3.5.4 Verificación Terraform en CI (obligatorio)	11
3.5.5 Ansible (obligatorio): propósito evaluable y reproducible	11
3.6 Pruebas no funcionales	11
3.6.1 Smoke tests (obligatorio)	11
3.6.2 Pruebas de contrato (obligatorio)	12
3.6.3 Carga ligera (obligatorio)	12
3.7 Hardening: seguridad y operación	12
3.7.1 Requests/Limits (obligatorio)	12
3.7.2 RBAC mínimo (obligatorio)	12
3.7.3 Secretos fuera del repositorio (obligatorio)	12
3.7.4 Notas de backup (obligatorio)	13
4. Entregables	13
4.1 Repositorio	13
4.2 Entregables mínimos (Fase 3)	13
5. Restricciones técnicas	13
6. Documentación	14
7. Consideraciones adicionales	14
7.1 Historial de desarrollo	14
7.2 Integridad académica	15

1. Resumen

En esta fase se entrega la versión **final operable** del sistema **EconoMarket**, construida como plataforma e-commerce modular con 6 **microservicios**, autenticación con **JWT**, integración con APIs externas con resiliencia, y procesos batch por **cron**.

No se redefine el dominio ni la arquitectura de Fases 1 y 2: se parte de que ya existen y funcionan. El objetivo ahora es completar lo que falta en la planificación y dejar el sistema **administrable, observable, endurecido y reproducible**, incluyendo **infraestructura como código verificable en CI**.

Requisitos evaluados explícitamente en esta fase: **panel administrativo (cupones y devoluciones), cron+notificaciones, hardening, pruebas no funcionales, IaC (Terraform+Ansible) con verificación en CI**.

2. Objetivos

2.1 Objetivo General

Operar EconoMarket como un sistema **administrable** (panel), **observable** (logs+métricas con dashboards/alertas), **endurecido** (seguridad/operación) y **reproducible** (IaC integral), con evidencia verificable y pipeline con checks.

2.2 Objetivos Específicos

Al finalizar la fase, el equipo debe ser capaz de:

1. Entregar un **panel administrativo funcional** para **cupones y devoluciones**.
2. Operar **cron/batch** y **notificaciones** (stock bajo, post-compra, recordatorios) con control anti-spam e idempotencia.
3. Implementar **observabilidad mínima** con **logs centralizados + métricas**, dashboards y alertas.
4. Aplicar **hardening**: requests/limits, RBAC mínimo, secretos fuera del repo y notas/procedimiento de backup.
5. Ejecutar **pruebas no funcionales**: smoke + contrato + carga ligera, con evidencia.
6. Levantar **toda la infraestructura** con **Terraform** y automatizar configuración con **Ansible**, ambos con verificación en CI.

3. Enunciado

3.1 Alcance específico de la Fase 3

Esta fase evalúa lo que queda pendiente según la planificación del curso: **admin + cron + hardening + pruebas no funcionales + IaC verificable**, manteniendo el

enfoque del proyecto (≥ 6 microservicios, JWT, FX, integraciones externas, cron y panel) y la operación real.

Regla de continuidad: todo lo de Fase 1 y 2 debe seguir funcionando: frontend, JWT end-to-end, despliegue y CI/CD.

3.2 Panel administrativo: cupones y devoluciones

3.2.1 Reglas generales del panel (obligatorio)

Debe existir un **panel web de administración** accesible solo por rol **admin**, coherente con el modelo de seguridad JWT ya existente (protección real de endpoints y uso desde UI).

Requisitos mínimos de panel

- Autenticación/Autorización:
 - Si no hay token válido $\rightarrow$ 401
 - Si no es admin $\rightarrow$ 403
- Trazabilidad: toda acción administrativa debe dejar evidencia (log estructurado) con `adminId`, acción, entidad afectada, timestamp.
- UX mínima: listado + búsqueda/filtros + detalle + crear/editar.

3.2.2 Cupones (obligatorio)

El sistema debe incluir cupones de descuento: **X% sobre el monto total**, agregados **antes del pago**, generados desde el panel admin, con **vencimiento** y límites de uso global y por cliente.

Modelo mínimo (recomendado)

- `code` (único; recomendado normalizar a upper-case)
- `percent` (1..100)
- `expiresAt` (datetime)
- `maxUsesGlobal` (int ≥ 1)
- `maxUsesPerCustomer` (int ≥ 1)
- `isActive` (bool)
- auditoría: `createdBy`, `createdAt`, `updatedAt`
- (recomendado) tabla/registro de redenciones:
`couponRedemptions(couponCode, customerId, orderId, redeemedAt)`

Reglas obligatorias de negocio

1. **Aplicación:** el cupón se valida y aplica en el flujo de check out **antes de pagar**.
2. **Vencimiento:** si `now > expiresAt` $\rightarrow$ cupón inválido.
3. **Usos globales:** no se puede exceder `maxUsesGlobal`.
4. **Usos por cliente:** un cliente no puede exceder `maxUsesPerCustomer`.

5. **Persistencia del descuento:** el descuento aplicado debe quedar persistido en la orden (para auditoría y para que el total no “cambie” después).
6. **Concurrencia:** el consumo de cupón debe ser atómico (evitar doble canje con solicitudes simultáneas).

Criterios de aceptación (cupones)

- Caso válido: total baja exactamente X%.
- Expirado/inactivo: error explícito.
- Sin usos globales: error explícito.
- Excede usos por cliente: error explícito.
- Evidencia: registro/contador por cliente y global verificable (DB/logs).

3.2.3 Devoluciones (obligatorio)

Debe existir flujo de devolución donde el cliente puede devolver **parcial o completamente** una compra seleccionando productos; el monto reembolsado se suma a la **cartera/wallet** del cliente; toda devolución debe ser **aprobada/rechazada** desde el panel admin; si se rechaza, se agrega comentario y se **notifica** al cliente el motivo.

Modelo mínimo

- `returnRequestId`
- `orderId, customerId`
- `items[]: {productId, qty, unitPriceAtPurchase, subtotal}`
- `status: PENDIENTE | APROBADA | RECHAZADA`
- `requestedAt`
- `reviewedAt, reviewedBy`
- `rejectionReason` (obligatorio si RECHAZADA)
- `approvedAmount` (si APROBADA)

Reglas obligatorias

1. **Validación de pertenencia:** sólo productos realmente comprados en esa orden.
2. **Validación de cantidad:** `qty devuelta <= qty comprada` (y si hay devoluciones previas, no exceder saldo disponible).
3. **Idempotencia:** no se puede “aprobar dos veces” y duplicar wallet.
4. **Wallet consistente:** al aprobar, se registra transacción en wallet con referencia a `returnRequestId` (para auditoría).
5. **Notificación:** al rechazar se envía email/notificación al cliente con el motivo.

Criterios de aceptación

- Demostrar devolución parcial y completa.
- Aprobar: wallet incrementa exactamente el monto aprobado.
- Rechazar: comentario persistente + notificación enviada al cliente.

3.3 Cron y notificaciones

La planificación exige cron y notificaciones (stock bajo / post-compra / recordatorios), y el enfoque del proyecto incluye cron para refrescos/recordatorios/batch.

3.3.1 Recordatorio de carrito (cron obligatorio)

Se recomienda explícitamente un cronjob para verificar y enviar recordatorios.

Requisitos

- Debe existir un **CronJob** (Kubernetes) que ejecute periódicamente:
 - Buscar carritos “inactivos” (por ejemplo, sin actividad > 24h) con ítems.
 - Enviar recordatorio al correo del cliente.
- Anti-spam: registrar `lastReminderAt` y no reenviar dentro de una ventana definida (ej. 24h).
- Idempotencia: si el cron se ejecuta dos veces seguidas, no debe duplicar recordatorios.

Evidencia mínima

- YAML del CronJob versionado.
- Logs de ejecución del cron (secuencia de runs + conteo de recordatorios enviados).

3.3.2 Notificación de stock bajo a proveedor (obligatorio)

Debe poder enviar notificaciones por correo a proveedores cuando un producto tenga ≤ 10 de stock.

Requisitos

- Disparador: evento al decrementar stock **y/o** cron verificador (aceptables ambos; documentar).
- Anti-spam por producto: no enviar repetidamente mientras permanezca bajo el umbral (ventana o estado de “ya notificado”).
- El correo debe indicar: producto/SKU, stock actual, umbral y acción sugerida (restablecer).

Evidencia mínima

- Caso demostrado: bajar stock a $\leq 10 \rightarrow$ correo emitido y log registrado.

3.3.3 Notificación post-compra al cliente (obligatorio)

Al finalizar compra debe enviarse un correo con productos, cantidades y total.

Requisitos

- Trigger: orden confirmada/pagada.

- Contenido mínimo: listado de productos + cantidad de cada uno + total de compra.
- Idempotencia por `orderId`: no reenviar el mismo correo.

Evidencia mínima

- Caso demostrado: compra completada → email enviado y trazable en logs.

3.4 Observabilidad: métricas, logs, dashboards y alertas

Debe implementarse observabilidad mínima: métricas y logs centralizados con dashboard.

Para fase final se requiere ELK + Prometheus + Grafana con requisitos específicos (índices por microservicio, Fluentd→Logstash, dashboards, alertas y metrics).

3.4.1 ELK Stack (obligatorio)

ElasticSearch

- Configurar un clúster para almacenar y buscar logs.
- Definir índices por microservicio (ej. `clientes-log`, `productos-log`, etc.).

Logstash

- Pipeline para recoger logs de contenedores (usando Fluentd) y transformarlos si es necesario.

Kibana

- Dashboards mínimos:
 - Errores por microservicio
 - Logs detallados de consultas a APIs
 - Tiempo promedio de respuesta por endpoint

Fluentd

- Debe capturar y enviar logs de aplicación y sistema a Logstash.

Requisito de formato de log (mínimo recomendado)

- Log estructurado (ideal JSON) con: `timestamp`, `level`, `service`, `method`, `path`, `status`, `durationMs`, `requestId` (recomendado), `error` (cuando aplique).

3.4.2 Prometheus (obligatorio)

Configurar Prometheus para recolectar métricas de microservicios.

Implementar exporter (ej. Node Exporter) para métricas de sistema (CPU/mem/red).

Definir alertas básicas: CPU/mem alto, fallo de microservicio, latencia alta.

Cada microservicio debe exponer endpoint `/metrics` compatible con Prometheus.

3.4.3 Grafana (obligatorio)

La etapa de despliegue debe:

- Conectar Grafana con Prometheus y ElasticSearch.
- Dashboards mínimos:
 - Estado general (uptime y uso de recursos)
 - Métricas por microservicio (latencia, tasa de solicitudes)
 - Logs críticos en tiempo real desde ElasticSearch
- Configurar alertas visuales y notificaciones.

3.5 Infraestructura como código (Terraform + Ansible)

La planificación establece explícitamente IaC I (Terraform) y II (Ansible), y exige que sea **ejecutable y verificable en CI** con estructura modular, variables, state, validation/plan; y en Ansible inventario, roles idempotentes y lint/syntax-check.

3.5.1 Regla principal (obligatoria)

Toda la infraestructura del entorno debe levantarse con Terraform, incluyendo:

- Red/base
- Cluster GKE
- Base de datos **fuera del cluster**
- Frontend en **Cloud Run**
- Componentes auxiliares necesarios (registry, state bucket, service accounts/IAM, etc.)

No se acepta infraestructura creada manualmente en consola como “paso oculto”. (Puede haber acciones manuales inevitables de credenciales/permisos, pero deben estar documentadas y el provisioning debe ser Terraform).

3.5.2 Terraform: infraestructura obligatoria

A) Red y seguridad base (obligatorio)

Terraform debe crear y parametrizar:

- VPC + subred(es) (región definida)
- reglas de firewall mínimas (abrir solo lo necesario)
- service accounts e IAM bindings mínimos

Criterio clave: la exposición pública debe ser la mínima necesaria (frontend + API pública). En fases anteriores ya existe regla de seguridad mínima de exposición de puertos (no exponer DB/Redis).

B) GKE (obligatorio)

Terraform debe crear:

- cluster GKE
- al menos 1 node pool (con configuración definida)
- outputs: nombre, región/zona, y datos necesarios para que CI/CD se conecte

C) Base de datos fuera del cluster (obligatorio)

Se mantiene DB **MS SQL Server** como requisito base.

Para fase final, la DB debe estar fuera del cluster y creada por Terraform. Opciones aceptables:

- **Cloud SQL for SQL Server** (si disponible en su cuenta/entorno)
- VM en Compute Engine con SQL Server (si Cloud SQL no es viable), siempre fuera de Kubernetes

Terraform debe incluir al menos:

- instancia (Cloud SQL o VM)
- configuración de red de acceso desde GKE (privada si es posible; si es pública, justificar y restringir)
- (recomendado) backups habilitados (y detallados en el runbook de backup)

D) Frontend en Cloud Run (obligatorio)

Terraform debe crear:

- servicio Cloud Run del frontend (imagen desde registry)
- variables de entorno necesarias (URL del gateway/API, etc.)
- IAM de acceso (público o controlado, justificar)

3.5.3 Estructura mínima del repo Terraform (obligatorio)

- `iac/terraform/modules/*` (mínimo 2 módulos; recomendado: `network`, `gke`, `database`, `cloudrun_frontend`, `registry`, `loadtest_vm`)
- `iac/terraform/environments/<env>` (aunque sea un solo env; documentar)
- variables parametrizadas (`project_id`, `region`, tamaños, nombres, etc.)
- outputs mínimos:
 - URL Cloud Run frontend
 - endpoint público de API/Ingress
 - datos del cluster (nombre/ubicación)
 - referencia de conexión a DB (sin passwords)

State

- Preferido: remote state en GCS (documentar).
- Si es local: justificar y explicar respaldo.

3.5.4 Verificación Terraform en CI (obligatorio)

La planificación exige validation/plan y checks en CI.

Jobs mínimos:

- `terraform fmt -check`
- `terraform validate`
- `terraform plan` (guardar log o artifact)

3.5.5 Ansible (obligatorio): propósito evaluable y reproducible

La planificación exige: inventario, roles, idempotencia, lint.

Para que sea evaluable y útil, Ansible debe configurarse sobre infraestructura creada con Terraform.

Requerimiento recomendado (estándar del curso para esta fase): VM de pruebas de carga + Locust

1. Terraform crea una VM `load-test`.
2. Ansible configura la VM con:
 - inventario (por IP o por nombre/metadata)
 - roles idempotentes (mínimo 2):
 - `base_tools` (python, pip, dependencias)
 - `locust` (instalación/configuración)
 - (recomendado) `test_runner` (scripts + ejecución headless + reportes)

Uso obligatorio del resultado

- La prueba de “carga ligera” de la sección 3.6 debe ejecutarse desde esta VM y generar un reporte.

Verificación Ansible en CI (obligatorio)

Jobs mínimos:

- `ansible-lint`
- `ansible-playbook --syntax-check`

3.6 Pruebas no funcionales

La planificación exige explícitamente: **smoke + contrato + carga ligera**.

3.6.1 Smoke tests (obligatorio)

Debe existir un script automatizado que valide, como mínimo:

- el frontend carga

- login (JWT)
- navegación mínima y flujo crítico: catálogo → carrito → checkout/orden

Requisitos

- Ejecutable desde CI o como comando reproducible.
- Produce salida clara (OK/FAIL) y logs para troubleshooting.

3.6.2 Pruebas de contrato (obligatorio)

Se mantienen contratos OpenAPI/JSON Schema versionados y validados automáticamente (pipeline falla ante contratos inválidos).

Requisitos

- Job CI “validate-contracts” (o equivalente).
- Evidencia: log/artefacto de validación.

3.6.3 Carga ligera (obligatorio)

Prueba de carga corta (1–3 min) sobre endpoints críticos.

- Recomendación: Locust desde VM `load-test` (Ansible), generando reporte.

Requisitos

- Parámetros mínimos: tasa de usuarios, duración, endpoints y umbrales.
- Reporte o salida reproducible (artefacto CI o archivo).

3.7 Hardening: seguridad y operación

La planificación define hardening mínimo: requests/limits, RBAC mínimo, secretos fuera del repo, notas de backup.

3.7.1 Requests/Limits (obligatorio)

- Todo Deployment crítico debe definir `resources.requests/limits`.
- Debe poder justificarse de forma básica (por ejemplo, en runbook/nota corta).

3.7.2 RBAC mínimo (obligatorio)

- No usar “cluster-admin para todo”.
- Definir ServiceAccounts y Role/RoleBinding con permisos mínimos.

3.7.3 Secretos fuera del repositorio (obligatorio)

- Prohibido versionar credenciales/tokens/keys (DB/JWT/registry).

- Uso de Secrets en K8s y variables seguras en CI.

3.7.4 Notas de backup (obligatorio)

- Documento breve: cómo respaldar y restaurar MSSQL (según DB elegida fuera del cluster).
- Debe incluir: frecuencia sugerida, ubicación del backup, pasos de restore.

4. Entregables

4.1 Repositorio

Entrega por enlace en UEDi (mismo esquema que fases previas).

Estructura sugerida (puede variar si se documenta):

- `services/<service-name>/...`
- `frontend/`
- `k8s/` (manifests app + observabilidad + cronjobs)
- `contracts/`
- `iac/terraform/`
- `iac/ansible/`
- `docs/`
- `README.md`
- `INTEGRANTES.md`

4.2 Entregables mínimos (Fase 3)

Para tener derecho a calificación, se debe cumplir al menos con:

- Panel admin: cupones + devoluciones (funcional).
- Cron + notificaciones: carrito, stock bajo, post-compra.
- Observabilidad completa solicitada (ELK + Prometheus + Grafana, dashboards y alertas, `/metrics`, Fluentd→Logstash).
- Hardening mínimo.
- Pruebas no funcionales (smoke + contrato + carga).
- IaC ejecutable y verificable en CI (Terraform + Ansible).
- Evidencias: repos IaC, logs CI, runbook y checklist de operación.

5. Restricciones técnicas

Se mantienen las restricciones base del proyecto:

- Nube: **Google Cloud**
- Backend: **TypeScript, Golang, Python**

- Frontend: frameworks soportados por **Vite**
- API interna: **gRPC**, excepto comunicación con frontend (REST)
- DB: **MS SQL Server**
- Seguridad: **JWT**
- Observabilidad: Prometheus, Grafana, ElasticSearch, Logstash, Kibana
- IaC: Terraform + Ansible
- Exposición de puertos: solo frontend y gateway/API pública; **no exponer Redis ni MSSQL públicamente**.

6. Documentación

En `/docs` deben incluirse evidencias y guías operativas (en el espíritu de fases anteriores: que un auxiliar pueda verificar sin adivinar).

Mínimos obligatorios

1. `runbook.md` (operación):
 - cómo desplegar y verificar
 - rollback/rollout
 - cómo ver dashboards Kibana/Grafana y targets Prometheus
 - cómo validar cronjobs y notificaciones
2. `iac.md`:
 - cómo ejecutar Terraform (init/plan/apply), variables requeridas, state
 - outputs importantes (URLs, endpoints)
 - cómo ejecutar Ansible (inventario, roles, idempotencia)
 - cómo correr Locust/carga ligera y dónde queda el reporte
3. `backup.md`:
 - backup/restore de MSSQL (DB fuera del cluster)
4. Evidencias de CI:
 - jobs Terraform (fmt/validate/plan)
 - jobs Ansible (lint/syntax-check)
 - validación de contratos
 - smoke + carga ligera
5. Evidencias de observabilidad (capturas o instrucciones reproducibles):
 - Kibana dashboards mínimos
 - Grafana dashboards mínimos
 - alert rules y un ejemplo de alerta

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** lunes 29 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.